

EXPERIMENT 5

Write a Python Program for the Missionaries and Cannibals Problem

Aim

To develop a Python program to solve the **Missionaries and Cannibals Problem** using the **Breadth-First Search (BFS)** algorithm and transport all missionaries and cannibals safely across the river while satisfying the given constraints.

Objective

- To understand the concept of state-space search in Artificial Intelligence.
- To implement the **Breadth-First Search (BFS)** algorithm.
- To find the shortest sequence of valid moves.
- To ensure all problem constraints are satisfied during the search.

Theory

The **Missionaries and Cannibals Problem** is one of the classical Artificial Intelligence search problems. Three missionaries and three cannibals are standing on one side of a river with a boat that can carry a maximum of two people. The objective is to transport everyone safely to the other side of the river.

The following constraints must always be satisfied:

- The boat can carry only one or two people.
- On either river bank, the number of cannibals should never exceed the number of missionaries unless there are no missionaries on that side.
- The boat cannot cross the river without at least one passenger.

The **Breadth-First Search (BFS)** algorithm is used because it explores all possible states level by level and guarantees the shortest valid solution.

Algorithm

Step 1: Represent the initial state as **(3, 3, L)**, where all missionaries and cannibals are on the left bank.

Step 2: Represent the goal state as **(0, 0, R)**, where everyone has crossed the river.

Step 3: Insert the initial state into a queue.

Step 4: Remove the first state from the queue.

Step 5: Check whether the current state is the goal state.

Step 6: If the goal is reached, display the solution and stop.

Step 7: Otherwise, generate all possible successor states by moving one or two people across the river.

Step 8: Check whether each generated state satisfies the constraints.

Step 9: Add all valid and unvisited states to the queue.

Step 10: Repeat Steps 4 to 9 until the goal state is reached.

Flowchart

Start



Initialize Initial State



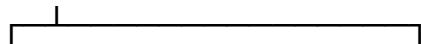
Insert Initial State into Queue



Remove First State



Is Goal State Reached?



Yes



Display

Solution



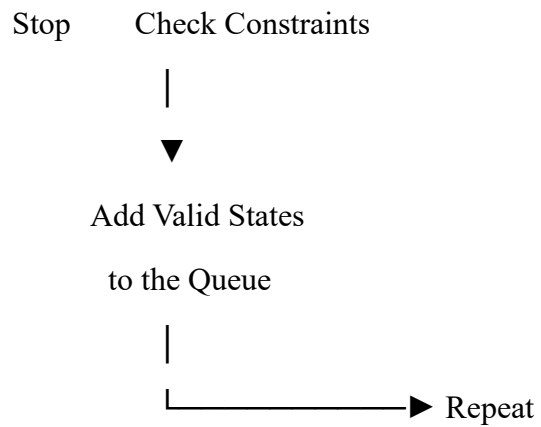
No



Generate Successor

States





Python Program

```
from collections import deque
```

```
def is_valid(m, c):
```

```
    if m < 0 or c < 0 or m > 3 or c > 3:
```

```
        return False
```

```
    if m > 0 and c > m:
```

```
        return False
```

```
    if (3 - m) > 0 and (3 - c) > (3 - m):
```

```
        return False
```

```
    return True
```

```
def bfs():
```

```
    start = (3, 3, 'L')
```

```
    goal = (0, 0, 'R')
```

```
    queue = deque([(start, [])])
```

```
    visited = set()
```

```
while queue:
```

```
    (m, c, boat), path = queue.popleft()
```

```
    if (m, c, boat) in visited:
```

```
        continue
```

```
    visited.add((m, c, boat))
```

```
    path = path + [(m, c, boat)]
```

```
    if (m, c, boat) == goal:
```

```
        return path
```

```
    moves = [(1,0), (2,0), (0,1), (0,2), (1,1)]
```

```
    for dm, dc in moves:
```

```
        if boat == 'L':
```

```
            new_state = (m-dm, c-dc, 'R')
```

```
        else:
```

```
            new_state = (m+dm, c+dc, 'L')
```

```
        if is_valid(new_state[0], new_state[1]):
```

```
            queue.append((new_state, path))
```

```
    return None
```

```
solution = bfs()
```

```
print("Solution Path:\n")
```

for state in solution:

```
print(state)
```

Sample Output

Solution Path:

(3, 3, 'L')

(3, 1, 'R')

(3, 2, 'L')

(3, 0, 'R')

(3, 1, 'L')

(1, 1, 'R')

(2, 2, 'L')

(0, 2, 'R')

(0, 3, 'L')

(0, 1, 'R')

(1, 1, 'L')

(0, 0, 'R')

Explanation

Each state is represented as:

(Number of Missionaries on the Left Bank, Number of Cannibals on the Left Bank, Boat Position)

Example:

- **(3, 3, L)** means all missionaries and cannibals are on the left bank.
- **(0, 0, R)** means everyone has safely reached the right bank.

The **Breadth-First Search (BFS)** algorithm explores all valid states level by level and guarantees the shortest sequence of moves while ensuring that missionaries are never outnumbered by cannibals on either bank.

Advantages

- Guarantees the shortest solution.
- Explores all valid states systematically.

- Suitable for state-space search problems.
- Easy to implement using a queue.

Applications

- Artificial Intelligence search problems.
- Robotics.
- Automated planning systems.
- State-space exploration.
- Puzzle-solving applications.

Result

The Python program successfully solved the **Missionaries and Cannibals Problem** using the **Breadth-First Search (BFS)** algorithm and generated the shortest valid sequence of moves while satisfying all the given constraints.

Conclusion

The **Missionaries and Cannibals Problem** demonstrates the application of **Breadth-First Search (BFS)** in solving state-space search problems. The algorithm systematically explores all valid states and guarantees the shortest solution while ensuring that the problem constraints are never violated. This experiment highlights the importance of **BFS** in Artificial Intelligence for solving planning and constraint-based problems.

```

r = 2
● PS C:\Users\Varshini M\AppData\Local\Programs\Microsoft VS Code> & "C:\U
/OneDrive/Desktop/Here!!!/ARTIFICIAL INTELLIGENCE/VSC EXPs/EXP1.py"
Solution Path:

(3, 3, 'L')
(3, 1, 'R')
(3, 2, 'L')
(3, 0, 'R')
(3, 1, 'L')
(1, 1, 'R')
(2, 2, 'L')
(0, 2, 'R')
(0, 3, 'L')
(0, 1, 'R')
(1, 1, 'L')
(0, 0, 'R')

```

int-based problems.

C:\Users\Varshini M\OneDrive\Desktop\Here!!!\ARTIFICIAL INTELLIGENCE\VSC EXPs> EXP1.py>...

```
1  from collections import deque
2
3  def is_valid(m, c):
4      if m < 0 or c < 0 or m > 3 or c > 3:
5          return False
6
7      if m > 0 and c > m:
8          return False
9
10     if (3 - m) > 0 and (3 - c) > (3 - m):
11         return False
12
13     return True
14
15 def bfs():
16     start = (3, 3, 'L')
17     goal = (0, 0, 'R')
18
19     queue = deque([(start, [])])
20     visited = set()
21
22     while queue:
23         (m, c, boat), path = queue.popleft()
24
25         if (m, c, boat) in visited:
26             continue
27
28         visited.add((m, c, boat))
29         path = path + [(m, c, boat)]
30
31         if (m, c, boat) == goal:
32             return path
33
34         moves = [(1,0), (2,0), (0,1), (0,2), (1,1)]
35
36         for dm, dc in moves:
37             if boat == 'L':
38                 new_state = (m-dm, c-dc, 'R')
39             else:
40                 new_state = (m+dm, c+dc, 'L')
41
42             if is_valid(new_state[0], new_state[1]):
43                 queue.append((new_state, path))
44
45     return None
46
47 solution = bfs()
48
49 print("Solution Path:\n")
50
51 for state in solution:
52     print(state)
53
```